DPS921 Final Cheatsheet

Answer bank + corrected code bank

Part A: Answer Bank

Week 8: Thrust

Q: Why does Thrust simplify GPU programming?
A: Thrust provides high-level STL-like algorithms such as `sort`, `transform`, `reduce`, and `scan`, so the programmer describes the operation instead of writing kernels, indexing, launches, and temp-memory handling by hand. This gives shorter code and faster development for common data-parallel tasks.

Q: What are Thrust's advantages and limits?
A: Advantages: concise code, reusable algorithms, easier debugging, and optimized implementations. Limits: less low-level control, possible temporary-memory overhead, and weaker fit for specialized fine-tuned kernels.

Q: Why does sorting help many parallel algorithms?
A: Sorting organizes data so searching, grouping, duplicate removal, counting, and range queries become easier. After sorting, algorithms can use binary search or bound queries instead of scanning every element, and equal keys become adjacent, which helps aggregation and grouping.

Q: How are sorting and searching used together in Thrust?
A: A common pattern is `thrust::sort` first, then `thrust::binary_search`, `lower_bound`, or `upper_bound` on the sorted range. This supports tasks such as testing membership, locating all equal keys, grouping records by key, or preparing data for reduction.

Q: What does a scan produce and where is it useful?
A: A prefix-sum or scan produces running totals. Example: input `[2,4,3,1]` gives inclusive output `[2,6,9,10]`. Scan is a fundamental building block for offsets, compaction, grouping, range-sum queries, histogram placement, and output positioning in parallel algorithms.

Week 9: CUDA Memory + Optimization

Q: What are threads, blocks, and grids in CUDA?
A: A thread is the smallest execution unit and usually handles one element. Threads are grouped into blocks; threads in the same block can share `__shared__` memory and synchronize with `__syncthreads()`. A grid is the full collection of blocks launched by one kernel call. Hardware executes threads in warps of 32.

Q: Why is block size important?
A: Block size affects occupancy, warp utilization, and scheduling efficiency. Good choices are usually multiples of 32, often 128, 256, or 512. Too few threads or blocks can leave GPU resources idle; very small blocks increase overhead; a poor block size may reduce occupancy or waste memory bandwidth.

Q: Compare global and shared memory.
A: Global memory is large and visible to all threads, but it has high latency. Shared memory is small, on-chip, and visible only to threads in the same block, but it is much faster. Shared memory acts like a programmer-managed cache and is most useful when the same data will be reused by several threads in a block.

Q: When does shared memory significantly help?
A: It helps when data loaded from global memory is reused multiple times, such as tiled matrix multiplication, convolution, stencil operations, reductions, or scans. It usually does not help much for a pure copy where each value is read once and written once, because there is no reuse to amortize the extra load/store and synchronization cost.

Q: What does asynchronous execution mean in CUDA?
A: The CPU can launch work without waiting for each earlier GPU operation to fully finish. Different streams may overlap kernels with transfers or run independent GPU tasks concurrently.

Q: Why can streams help, and why might they not?
A: Streams help when transfers and kernels are independent and there is enough work and hardware capacity to overlap them. They may not help much if tasks are small, dependent on each other, limited by one resource, or if stream-management overhead outweighs the benefit.

Week 10: OpenMP + Race Conditions

Q: What happens when execution enters and exits a parallel region?
A: OpenMP uses fork-join. One thread starts, entering `\#pragma omp parallel` creates a team, the team executes concurrently, then threads synchronize and join back to one thread.

Q: What is the difference between shared and private variables?
A: A shared variable is one memory location visible to all threads. A private variable gives each thread its own independent copy. Scope matters because wrong sharing can cause race conditions and wrong results, while excessive sharing also hurts performance due to synchronization or contention.

Q: Why is reduction preferred for sums?
A: A reduction creates a private copy of the variable for each thread, allows threads to accumulate locally without contention, and combines the partial results at the end. This gives correct results and usually scales better than `atomic` or `critical`, which synchronize more often.

Q: Manual vs implicit partitioning in OpenMP?
A: Manual partitioning means the programmer computes `start/end` ranges for each thread. It gives control but requires more code and can miss leftover iterations. Implicit partitioning with `\#pragma omp parallel for` lets OpenMP divide loop iterations automatically, which is simpler and less error-prone, though it provides less exact control over work distribution.

Week 11: Loop Parallelism + Performance

Q: Why do dependencies prevent direct loop parallelization?
A: Parallel iterations must be independent. If iteration `i` needs a value from `i-1`, such as `A[i] = A[i-1] + 5;`, simultaneous execution can read unfinished values or violate required order.

Q: Outer loop, inner loop, or collapse?
A: Parallelizing the outer loop is usually best because it has lower scheduling overhead and better locality. Parallelizing the inner loop is useful only when the outer loop has too few iterations. `collapse(2)` merges nested loops into one larger iteration space and improves load balance when the outer loop alone does not expose enough parallel work.

Q: Static, dynamic, and guided scheduling?
A: `static` preassigns fixed chunks and is best for regular workloads with similar iteration cost. `dynamic` hands out chunks at runtime and is better for irregular workloads, but with higher overhead. `guided` starts with larger chunks and gradually shrinks them, trying to balance overhead and load balance for large loops.

Q: What is false sharing?
A: False sharing happens when threads update different variables that happen to sit on the same cache line. Correctness may remain fine, but cache-line invalidations cause heavy slowdown. Fixes include using private local variables, writing shared results less often, or padding per-thread data so each thread writes to a separate cache line.

Week 12: Design Patterns + Model Comparison

Q: SPMD vs MPMD?
A: SPMD means all workers run the same program on different data. MPMD means different workers may run different programs on different data. MPI supports MPMD naturally because separate processes can run separate executables; OpenMP is more naturally SPMD because all threads live inside one shared-memory program.

Q: How can matrix multiplication use the same decomposition with OpenMP, MPI, or CUDA?
A: The decomposition idea stays the same: split the output matrix by rows, columns, or blocks. What changes is the execution model. OpenMP maps subproblems to shared-memory CPU threads, MPI maps them to separate processes that exchange data explicitly, and CUDA maps them to many GPU threads or tiles. Same partitioning logic, different mapping and communication mechanism.

Q: How is task parallelism implemented in OpenMP, MPI, and CUDA?
A: OpenMP tasks are scheduled by the runtime onto threads. MPI uses separate worker processes with explicit messages. CUDA launches kernels where many GPU threads each execute a small piece of the task.

Q: How do you choose between MPI, OpenMP, and CUDA for an application?
A: Match the hardware and decomposition: distributed multi-node simulation fits MPI+SPMD, shared-memory workstation image processing fits OpenMP loop parallelism, and large matrix/ML GPU work fits CUDA+SPMD.

Week 13: Load Balancing + Scalability + Debugging

Q: Why is load balancing important?
A: Parallel runtime is usually determined by the slowest processor or thread. If one worker gets much more expensive work, others finish early and wait idle.

Q: Why does equal task count not guarantee balanced execution?
A: Tasks can have very different costs. Two processors may each receive four tasks, but one set may be much heavier. For irregular workloads, balance must consider task cost, not just task count, which is why dynamic scheduling or dynamic load balancing is often better.

Q: Why doesn't adding processors always improve performance?
A: Scalability is limited by serial work, communication overhead, synchronization overhead, load imbalance, lock contention, cache contention, false sharing, and memory bandwidth limits. Once overhead grows faster than useful computation shrinks, adding processors gives diminishing returns or can even slow the program down.

Q: How can MPI scalability be improved when communication becomes a bottleneck?
A: Use `MPI_Isend()` and `MPI_Irecv()` to overlap communication with computation. Also reduce boundaries, combine small messages into fewer larger ones, and reduce synchronization frequency.

Q: What is a race condition and why is it hard to reproduce?
A: A race condition occurs when multiple threads or processes access shared data without correct synchronization and at least one access is a write. The result depends on timing and scheduling, which vary from run to run, so the bug can appear intermittently. Fixes include `atomic`, `critical`, locks, reductions, or redesigning to reduce shared updates.

Q: How do deadlock and lock order relate?
A: Deadlock occurs when threads hold resources while waiting forever for each other, such as one thread locking A then waiting for B while another locks B then waits for A. A standard fix is to acquire locks in a consistent global order and always release them after the protected section.

Metric	Formula	Meaning
Speedup	$S = T_1/T_p$	How much faster parallel is
Efficiency	$E = S/p$	Fraction of hardware used well
Comm. %	$C/T \times 100$	Portion of runtime spent communicating
Load-balance rule	Runtime is max(processor workloads)	
Static vs dynamic	Equal-cost work: static; irregular work: dynamic	
Mandelbrot	Dynamic scheduling/load balancing is preferred	

Part B: Code Bank

Thrust Patterns

Sort + Search + Filter

```
#include <thrust/device_vector.h>
#include <thrust/host_vector.h>
#include <thrust/sort.h>
#include <thrust/copy.h>
#include <thrust/binary_search.h>

struct IsEven {
    __host__ __device__
    bool operator()(int x) const { return x % 2 == 0; }
};

thrust::device_vector<int> v{7,2,9,4,1,8,6};
int target = 4;
thrust::sort(v.begin(), v.end());
bool found = thrust::binary_search(v.begin(), v.end(), target);
thrust::device_vector<int> even(v.size());
auto endIt = thrust::copy_if(v.begin(), v.end(), even.begin(), IsEven());
even.erase(endIt, even.end());
thrust::host_vector<int> hv = v;
thrust::host_vector<int> he = even;
```

Explicit GPU Policy

```
#include <thrust/reduce.h>
#include <thrust/execution_policy.h>

thrust::device_vector<int> data(1000, 1);
int sum = thrust::reduce(thrust::device, data.begin(), data.end(), 0);
```

Counting Iterator + Inclusive Scan + Stream

```
#include <cuda_runtime.h>
#include <thrust/device_vector.h>
#include <thrust/scan.h>
#include <thrust/iterator/counting_iterator.h>
#include <thrust/execution_policy.h>

int N = 6;
cudaStream_t s;
cudaStreamCreate(&s);
thrust::device_vector<int> out(N);
auto first = thrust::make_counting_iterator(1);
auto last  = first + N;
thrust::inclusive_scan(thrust::cuda::par.on(s), first, last, out.begin());
cudaStreamSynchronize(s);
thrust::host_vector<int> h = out;
cudaStreamDestroy(s);
```

Thrust Reminder

Use `thrust::device` for explicit device execution and `thrust::cuda::par.on(stream)` for stream-specific async execution.

CUDA Patterns

1D Kernel Index + Bounds

```
__global__ void kernel(int* A, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) A[i] = A[i] * 2;
}

int blockSize = 256;
int gridSize = (N + blockSize - 1) / blockSize;
kernel<<<gridSize, blockSize>>>(d_A, N);
```

2D Matrix Addition

```
__global__ void addMatrix(const int* A, const int* B, int* C, int M, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < M && col < N) {
        int idx = row * N + col;
        C[idx] = A[idx] + B[idx];
    }
}

dim3 block(16, 16);
dim3 grid((N + block.x - 1) / block.x,
          (M + block.y - 1) / block.y);
addMatrix<<<grid, block>>>(d_A, d_B, d_C, M, N);
```

Shared-Memory Copy Kernel

```
__global__ void copyKernel(const int* input, int* output, int N) {
    __shared__ int temp[256];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tid;

    if (i < N) temp[tid] = input[i];
    __syncthreads();
    if (i < N) output[i] = temp[tid];
}
```

Shared Memory Rule

Load from global once, synchronize, then reuse. Helpful when values are reused; weak benefit for simple copy-only kernels. `__syncthreads()` is required when threads read data written by other threads in the block.

Memory + Warp Reminders

Registers: fastest/private. Shared: fast/block-local. Constant: read-only, best when threads read same address. Global: largest/slowest. Coalescing = adjacent threads, adjacent addresses. Divergence = one warp, different branches.

OpenMP Patterns

Parallel Region

```
#include <omp.h>

#pragma omp parallel
{
    int id = omp_get_thread_num();
    int nt = omp_get_num_threads();
}
```

Reduction for Sum

```
long long sum = 0;
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++) {
    sum += A[i];
}
```

Manual Partitioning

```
long long sum = 0;
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int nt = omp_get_num_threads();
    int chunk = N / nt;
    int start = id * chunk;
    int end = (id == nt - 1) ? N : start + chunk;
    long long localSum = 0;

    for (int i = start; i < end; i++) localSum += A[i];

    #pragma omp atomic
    sum += localSum;
}
```

Nested Loop + Collapse

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        A[i][j] = i + j;
    }
}
```

Scheduling

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) { ... }

#pragma omp parallel for schedule(dynamic, 4)
for (int i = 0; i < N; i++) { ... }

#pragma omp parallel for schedule(guided)
for (int i = 0; i < N; i++) { ... }
```

Dependency Example: Not Directly Parallel

```
for (int i = 1; i < N; i++) {
    A[i] = A[i - 1] + B[i];
}
```

False Sharing Fix

```
struct alignas(64) PaddedSum { long long value; };
PaddedSum partial[8];

#pragma omp parallel num_threads(8)
{
    int id = omp_get_thread_num();
    long long local = 0;
    for (int i = 0; i < N; i++) local += i;
    partial[id].value = local;
}
```

Debug Checklist

Check	Typical fix
Shared sum race	Use `reduction`, `atomic`, or `critical`
MPI tag mismatch	Sender and receiver tags must match
Wrong `cudaMemcpy` direction	Host to device before kernel, device to host after
Missing bounds check	Add `if (i < N)` or 2D equivalent
Missing `__syncthreads()`	Add after shared-memory load when data is reused
False sharing	Use local vars or padding
Deadlock	Consistent lock order, always unlock
Equal tasks but poor scaling	Consider task cost and dynamic scheduling

App	Pick	Why
Weather sim	MPI+SPMD	Multi-node grid + boundary exchange
Image processing	OpenMP	Independent pixel/loop work
Matrix/ML ops	CUDA+SPMD	Many identical GPU computations